Strings in depth

Why string methods hand back new strings, the everyday methods with their real results, and split and join.

THE QUESTION

How do you tidy, search, cut up and rebuild text, such as a line a person typed?

THE INTUITION

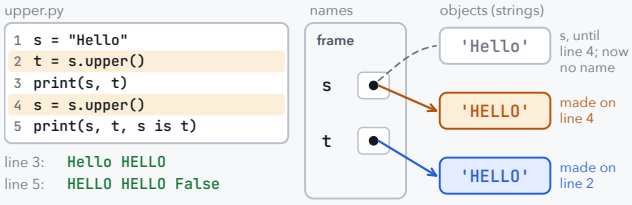
A string is a row of characters that never changes. A method reads it and hands back something new: a string, a number, True or False, or a list. To keep the result, give it a name.

KEY TAKEAWAY

Write `s = s.strip()`, not just `s.strip()`. `split` turns a string into a list of strings and `sep.join(...)` turns that back into one string.

raw text → tidy: strip, lower → search: in, find → cut: split → rebuild: join → a new string

1 Methods hand back a new string



A string is **immutable**: its characters never change (the slicing sheet showed `w[0] = "J"` failing). A **method**, a function called with a dot (see the lists sheet), such as `upper()`, **returns** a new string and leaves `s` alone. Line 4 points `s` at another new string; `s is t` is False: equal text, two objects. A bare `s.upper()` changes nothing: `print(s)` still shows `Hello`.

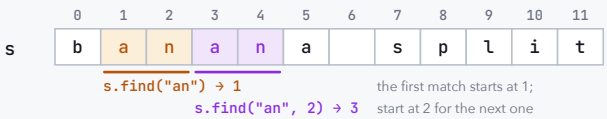
2 Change case, trim the ends, replace

```
w = "hello World"
w.upper() → 'HELLO WORLD'
w.lower() → 'hello world'
w.title() → 'Hello World'
w.capitalize() → 'Hello world'
w.replace("o", "0") → 'hell0 W0rld'
w.replace("o", "0", 1) → 'hell0 World'
```

```
p = "  hi  "    = one space
p.strip() → 'hi'
p.lstrip() → 'hi  '
p.rstrip() → '  hi'
"**hi!**".strip("*") → 'hi!'
p.strip().upper() → 'HI'
w → 'hello World'
```

A tinted box is a space inside a string. `strip()` removes **whitespace** (spaces, tabs, newlines) from the two ends, never the middle; give it characters and it strips those instead. `replace(old, new)` swaps every match, or only the first `n` with a third number. The result is a string, so another method can follow it.

3 Search with `in`, `find`, `index`, `count`




```
"an" in s → True
s.count("an") → 2
s.find("x") → -1
"AN" in s → False
s.startswith("ban") → True
s.index("x") → ValueError: substring not found
```

`sub in s` asks whether `sub` is a **substring**, a run of neighbouring characters of `s`; capitals count. `find` gives the index where the first match starts, or −1 if there is none; `index` raises a **ValueError** instead. `count` counts matches that do not overlap; `startswith` and `endswith` check the ends.

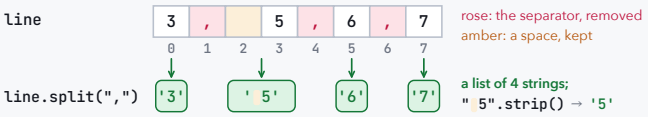
4 The `is` methods test every character

string →	"42"	"4.2"	"-7"	"abc"	"HI 5"	" "	""
.isdigit()	True	False	False	False	False	False	False
.isalpha()	False	False	False	True	False	False	False
.isspace()	False	False	False	False	False	True	False
.islower()	False	False	False	True	False	False	False
.isupper()	False	False	False	False	True	False	False

True only when every character passes and there is at least one; islower and isupper look only at the letters, and need at least one letter

Each gives True or False. `"-7".isdigit()` is False although `int("-7")` gives -7, so `if t.isdigit():` before `int(t)` lets through only whole numbers from 0 up. The **empty string** `""` passes no test. `isspace()` catches a line of only spaces.

5 `split` cuts a string into a list




```
"a b c".split() → ['a', 'b', 'c']
"a b c".split(" ") → ['a', '', 'b', 'c']
"2026-09-27".split("-", 1) → ['2026', '09-27']
"alice.smith@example.com".split("@") → ['alice.smith', 'example.com']
```

`s.split(sep)` cuts at every `sep`, drops it, and returns the pieces as a **list of strings**. A space beside a comma stays, as in `' 5'`. The notebook adds up such input: `for part in line.split(","):` with `total += int(part.strip())` gives 21. With no argument, `split()` gives the words of a line; a second number caps the cuts.

6 `join` glues a list back into one string

```
y = ['a', 'e', 'l', 'l', 'o']      "-".join(y): 4 dashes between 5 items, none at the ends
['a'] - ['e'] - ['l'] - ['l'] - ['o'] ⟶ 'a-e-l-l-o'
```

```
"".join(y) → 'aello'
", ".join(["Alice", "Bob", "Carol"]) → 'Alice, Bob, Carol'
"-".join("abc") → 'a-b-c'
",".join([1, 2, 3]) → TypeError:
sequence item 0: expected str instance, int found
",".join(map(str, [1, 2, 3])) → '1,2,3'
```

`sep.join(items)` is the reverse of `split`: the separator comes first, the list goes in the brackets, and out comes one string. Every item must be a string, so numbers give a **TypeError**; convert each with `str()` in a loop, or with `map(str, nums)` (see the functions are values sheet).

Characters are numbers underneath

One character at a time, `ord` and `chr`, a Caesar shift, the order of strings, and building a string from pieces.

7 `for ch in s` visits every character

vowels.py

```
s = "Oak tree"
count = 0
for ch in s:
    if ch.lower() in "aeiou":
        count += 1
print(count)
```

prints 4

the space gets a pass too (row 4)

pass	ch	ch.lower()	in "aeiou"	count
1	'O'	'o'	True	1
2	'a'	'a'	True	2
3	'k'	'k'	False	2
4	' '	' '	False	2
5	't'	't'	False	2
6	'r'	'r'	False	2
7	'e'	'e'	True	3
8	'e'	'e'	True	4

A `for` loop over a string (see the for loops sheet) gives one character per pass, spaces included. `ch.lower()` lets `'O'` count; `in "aeiou"` asks whether that one character is among the five. The notebook's `while i < len(s):` with `s[i]` walks by index and also gets 4.

8 `ord` gives a code, `chr` a character

' '	'0'	'1'	'9'	'A'	'B'	'Z'	'a'	'b'	'z'
32	48	49	57	65	66	90	97	98	122

`ord("a") - ord("A")` → 32

`chr(ord("c") + 1)` → `'d'`

```
msg = "meet by the zoo"
k = 3
out = ""
for ch in msg:
    if ch.isalpha():
        n = (ord(ch) - 97 + k) % 26
        ch = chr(97 + n)
    out += ch
print(out)
```

one letter, 'y', with k = 3

```
ord('y') - 97    → 24
+ k              → 27
% 26             → 1
chr(97 + 1)      → 'b'
```

97 is ord('a'); past 'z' wraps to 'a'

prints `'phhw eb wkh crr'`

Each character has a number, its **code** (**Unicode**; **ASCII** for English letters and digits). `ord(ch)` gives the code, `chr(n)` the character. Letters run in order, so adding to a code shifts a letter. A **Caesar shift** moves each letter `k` places; `% 26` wraps past `'z'`; `k = -3` undoes it.

9 Strings compare character by character

`"apple" < "apricot"` → True

a	p	p	l	e		
a	p	r	i	c	o	t

'p' is 112

'r' is 114

`"Zebra" < "apple"` → True

Z	e	b	r	a
a	p	p	l	e

'Z' is 90

'a' is 97

`"10" < "9"` → True

1	0
9	

'1' is 49

'9' is 57

`"app" < "apple"` → True

a	p	p		
a	p	p	l	e

runs out first, so it is smaller

grey: equal so far · amber: the first difference decides · dashed: no character

`"Zebra".lower() < "apple".lower()` → False `int("10") < int("9")` → False

`<` on strings is **lexicographic**: the first position where they differ decides, by code; a string that runs out first is smaller. Capitals come before small letters, and digits compare as text: `max(["10", "9", "100"])` is `'9'`. Compare `lower()` versions, or `int`s.

10 Collect the pieces in a list, then join once

build.py

```
words = ["to", "be", "or"]
s = ""
for w in words:
    s = s + " " + w
print(repr(s))
```

prints `' to be or'` a space too many

w	s after the pass (a new string each time)
start	''
'to'	' to'
'be'	' to be'
'or'	' to be or'

the join version: `'to be or'`

squares.py: collect the pieces in a list, join once at the end

```
parts = []
for n in [1, 2, 3, 4]:
    parts.append(str(n * n))
print(" + ".join(parts))
```

prints

```
1 + 4 + 9 + 16
```

`s = s + " " + w` builds a new, longer string on every pass (a string cannot grow in place) and puts a space before the first word. `" ".join(words)` puts it only between. With pieces from a loop, `append` them to a list and join once. For a short result `+=` is fine.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints, or what you see; why, and the fix
`name.upper()`	`prints: alice` the new string was thrown away; `name = name.upper()` prints `ALICE`
`if s.find("a"):`	`prints: not found` `find` gives 0, the index of `'a'`, and 0 counts as False; `if "a" in s:` prints `found`
`", ".join([4, 5])`	`TypeError: sequence item 0: expected str instance, int found` `join` takes strings only: `", ".join(["4", "5"])` gives `4, 5`
`words.join(" ")`	`AttributeError: 'list' object has no attribute 'join'` the separator comes first: `" ".join(words)` gives `a b`
`input() < "9"`	`prints: True` typed 10: text compares by characters and `'1' < '9'`; `int(x) < 9` gives False

12 Try it (answers in green)

1. `"Hi There".lower().split()`?
   `['hi', 'there']`
2. `"banana".count("ana")`?
   1: matches do not overlap.
3. `":".join("a b  c".split())`?
   `'a:b:c'`
4. `"Zoo" < "apple"`, `"Apple" < "apple"`, `"12" < "3"`?
   True, True, True: codes decide.
5. `chr(ord("a") - 32)`?
   `'A'`: the capital is 32 codes lower.
6. Count the digits in `"R2-D2 x10"` with a `for` loop.
   4: `if ch.isdigit(): n += 1`

strings never change · keep what a method returns · split makes a list, join a string · compare by code

Source: the strings notebook, nipunbatra.github.io/comp22; the e-mail address is a placeholder. Every output and error message comes from running the code with Python 3.11.11. Next: the functions sheet.